

Modelling Distributed Systems Software & System Architectures

Distributed Systems

Andrea Omicini
andrea.omicini@unibo.it

Dipartimento di Informatica – Scienza e Ingegneria (DISI)
ALMA MATER STUDIORUM – Università di Bologna

Academic Year 2025/2026



- 1 Prologue
- 2 Software Architectures
- 3 Architectural Styles
- 4 Conclusion



Next in Line...

- 1 Prologue
- 2 Software Architectures
- 3 Architectural Styles
- 4 Conclusion



Our Motivation Here

Modelling distributed systems

- our basic ontology has just given us a ground over which we can start
- we need to learn a *general* way to represent distributed systems
- independently of their heterogeneous nature

Modelling component & systems in time & space

- how can distributed systems be *represented at design time*?
- how can distributed systems be *represented at run time*?
- how do we account for their *evolution over time*?
- how do we account for their *distribution in space*?

Disclaimer

- the following slides borrow a great deal from the slides coming with the book adopted as the basic one for this course, [Tanenbaum and van Steen, 2017]
- material from those slides (including pictures) has been re-used in the following, and integrated with new material according to the personal view of the professor
- every problem or mistake contained in these slides, however, should be attributed to the sole responsibility of this course's professor
- the goal for students, here, is to be exposed to the classical abstract view on distributed systems provided by the notions of *software* and *system architecture*

Next in Line...

- 1 Prologue
- 2 Software Architectures**
- 3 Architectural Styles
- 4 Conclusion



Software Architectures to Handle Complexity

Distributed systems are complex

- in order to manage their intrinsic complexity, distributed systems should be properly *organised*
- organisation of a distributed system is mostly expressed in terms of its *software components*

Software architectures expresses component organisation

- there are many ways to organise components of a distributed system, classified as **software architectures**
- there are many possible *instantiations* of a software architecture, where components have their actual *place* in the distributed system—often called **system architectures**

Architectural Styles to Classify Systems

An *architectural style* is formulated in terms of. . .

- *components*
- the way in which components are *connected* to each other
- the *data* flowing through the components
- the way in which all the above things are *configured* altogether to build the system

The notion of architectural style. . .

- encompasses a way to cluster and classify groups of similar systems—that is, having the *same* sort of *organisation*
- allow distributed systems to be *compared*
- but also provide general *patterns* for their overall *design*

Components & Connectors I

Components

- a **component** is a modular unit with well-defined *interfaces*
- which is *replaceable* within its environment
- interfaces are both *required* and *provided*—both ways, then

Connectors

- a **connector** is an abstraction *mediating* communication, coordination, cooperation among components
- that is, anything providing a *mechanism for interaction* among components

Components & Connectors II

Putting together components and connectors. . .

- produces a huge range of possible organisations and configurations
- that are then classified in terms of architectural styles



What is a Software Architecture?

Software architecture

*A **software architecture** is an abstraction of the run-time elements of a software system during some phase of its operation. A system may be composed of many levels of abstraction and many phases of operation, each with its own software architecture.* [Fielding, 2000]

Architectural elements

*A software architecture is defined by a configuration of **architectural elements** – components, connectors, and data – constrained in their relationships in order to achieve a desired set of architectural properties.* [Fielding, 2000]

Architectural Elements

Components

A **component** is an abstract unit of software instructions and internal state that provides a transformation of data via its interface

Connectors

A **connector** is an abstract mechanism that mediates communication, coordination, or cooperation among components

Data

A **datum** is an element of information that is transferred from a component, or received by a component, via a connector

Architectural Properties & Constraints

Architectural properties

The set of **architectural properties** of a software architecture is derived from the *selection* and *arrangement* of components, connectors, and data within a system

- *functional properties*
- *quality attributes* such as ease of evolution, reusability of components, efficiency, and dynamic extensibility

Properties are induced by the *set of constraints* within an architecture

Architectural constraints

Architectural constraints are often motivated by the application of a software engineering principle to an aspect of the architectural elements

Next in Line...

- 1 Prologue
- 2 Software Architectures
- 3 Architectural Styles**
- 4 Conclusion



Architectural Styles

Architectural Style

*An **architectural style** is a coordinated set of architectural constraints that restricts the roles/features of architectural elements and the allowed relationships among those elements within any architecture that conforms to that style*^[Fielding, 2000]

Architectural styles

- are a mechanism for categorising architectures and defining their common characteristics
- provide an abstraction for the interactions of components, capturing the essence of a pattern of interaction by ignoring the accidental details of the rest of the architecture

Main Examples of Architectural Styles

Identification of architectural styles

- **architectural styles** – like patterns in software engineering – are to be *devised out* rather than invented
- today, four different architectural styles have been identified as the main ones for distributed systems

Main architectural styles for distributed systems

- **layered** architectures
- **object-based** architectures
- **data-centered** architectures
- **event-based** architectures
- + **shared data-space** architectures

Layered Architectures I

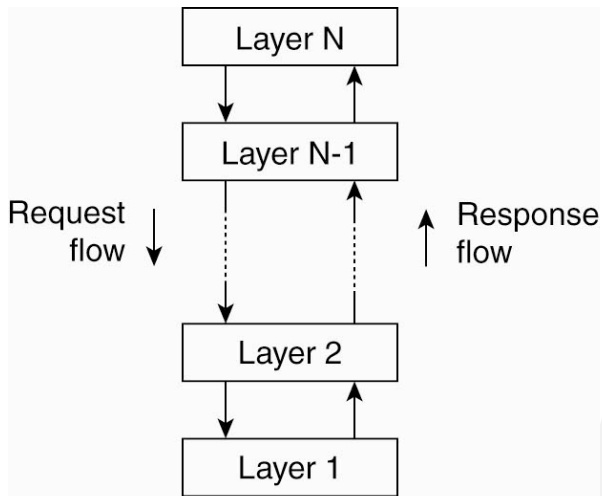
Basic idea

- components are organised in a *layered fashion*
- where components of a layer *only* call components of the layer below, and are *only* called by the components of the layer above

Data flow

- the request-response flow is always top-down / bottom-up
- control flow follows the same pattern along with data

Layered Architectures II



[Tanenbaum and van Steen, 2017]

Object-based Architectures I

Basic idea

- components are *objects*
- components are connected through a *RPC mechanism*

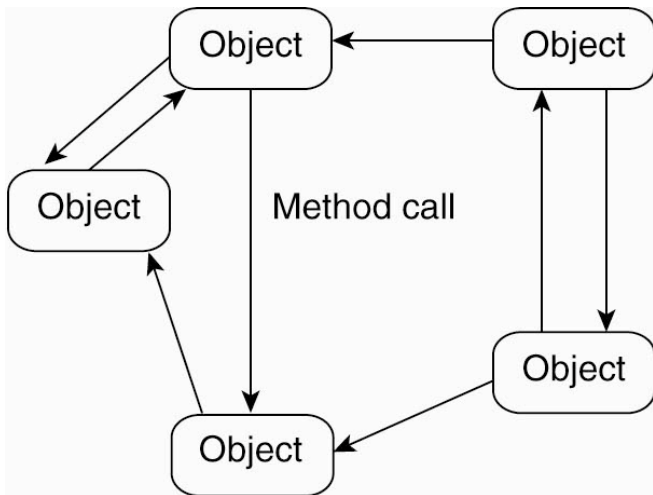
Client-server architectures

- ... are built out of this style

Layered and object-based architectures

- ... are the most important styles for distributed systems today
- however, a lot of things are going to happen in the future, which may change such an overall picture

Object-based Architectures II



[Tanenbaum and van Steen, 2017]

Data-centred Architectures I

Basic idea

- communication among processes occurs through a **shared repository**
- the repository might be either passive (reactive) or (pro)active

Main features

- ... depends on the choice made for the shared repository
 - how *information* is represented
 - how *events* are handled
 - how the *shared repository* behave in response to interaction
 - how *processes* interact with / through the shared repository

Data-centred Architectures II

Examples are everywhere

- web-based systems, for instance, are largely data-centric
- also, many distributed applications still work by sharing files around the network



Event-based Architectures I

Basic idea

- processes communicate through an **event bus**
- through which *events* are propagated
- possibly carrying *data* along

Main example: Publish/subscribe systems

- *publishers* publish events through the middleware
- *subscribers* receive events to which they have subscribed

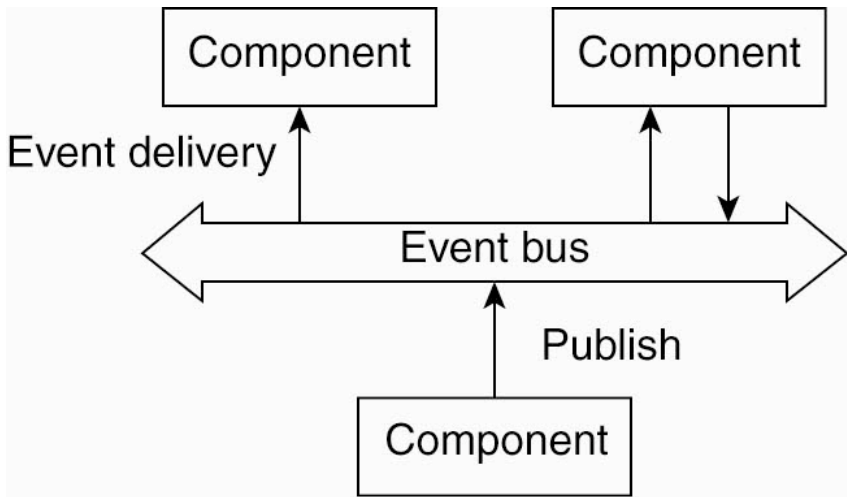
Event-based Architectures II

Main feature

- processes can communicate with no need to *reference* each other / to know each other—they are *referentially uncoupled*
- processes can communicate with no need to *share* the same space—they are *uncoupled in space*



Event-based Architectures III



[Tanenbaum and van Steen, 2017]

Shared Data-space Architectures I

Basic idea

- putting together Data-centric and Event-based architectures
- the shared repository is a shared persistent data-space, and also an event bus
- where data is stored and accessed
- along with related events

Main example: Blackboard systems

- processes put data in the blackboard
- the blackboard aggregates *knowledge*, implements *policies*, and drive the *coordination* of processes

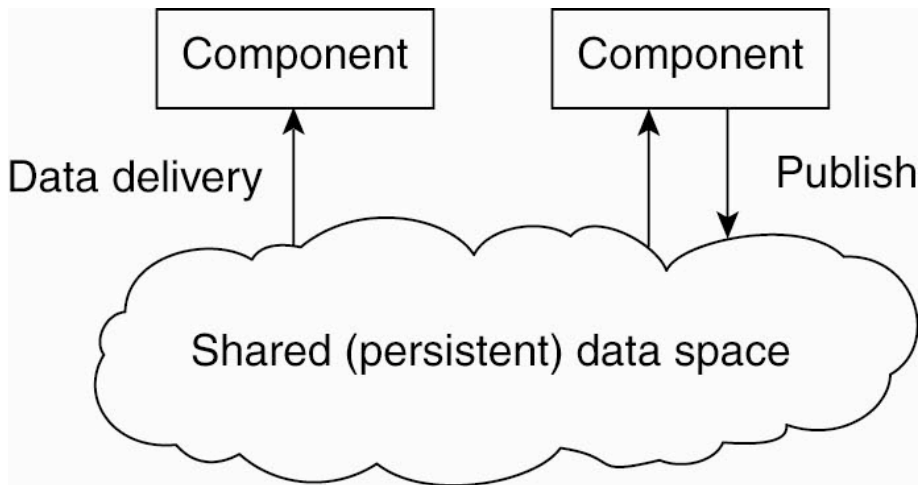
Shared Data-space Architectures II

Main feature

- processes can communicate with no need of compresence
- processes are also *uncoupled in time*



Shared Data-space Architectures III



[Tanenbaum and van Steen, 2017]

Next in Line...

- 1 Prologue
- 2 Software Architectures
- 3 Architectural Styles
- 4 Conclusion**



Summing Up I

Organisation of distributed systems

- architectural styles deal with software organisation
- they are approximative and maybe non-scientific ways to model systems
- however they are expressive and abstract enough to help distributed system engineering

Software vs. system architectures

- software architectures are concerned with logical organisation
 - possibly over time
- system architectures are concerned with component placement in a distributed setting
 - they deal with spatial distribution

Questions

- this looks like a simple & solid start for the discipline of distributed system engineering
- is this enough for a well-grounded foundation for a science of computational distributed systems?
 - e.g., can we prove theorems based on software & system architectures?
- what is the role of “math-like” formalism for that, such as process algebras?



Modelling Distributed Systems Software & System Architectures

Distributed Systems

Andrea Omicini
andrea.omicini@unibo.it

Dipartimento di Informatica – Scienza e Ingegneria (DISI)
ALMA MATER STUDIORUM – Università di Bologna

Academic Year 2025/2026



References

[Fielding, 2000] Fielding, R. T. (2000).

Architectural Styles and the Design of Network-based Software Architectures.

PhD thesis, University of California, Irvine, CA, USA

(APICe) <http://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm>

[Tanenbaum and van Steen, 2017] Tanenbaum, A. S. and van Steen, M. (2017).

Distributed Systems. Principles and Paradigms.

Pearson Prentice Hall, 3rd edition

<https://www.distributed-systems.net/index.php/books/ds3/>

